Assignment 4 | Facade Pattern

Berdibek Meiirbek | SE-2528 | Option B: Online Order Processing

1. Introduction

An order needs stock checks, payment, shipping, and a confirmation. Calling these services directly would make the client responsible for their order and for undoing completed steps. OrderFacade exposes one method, placeOrder(cart, customerId), and coordinates four subsystem classes [1, 2].

The services simulate a shop in memory. InventoryService calculates the total and changes stock; PaymentProcessor changes balances; ShippingService stores shipment records; NotificationService formats and records confirmations. These are state changes, not only printed messages. Amounts are stored as integer cents.

2. UML class diagram

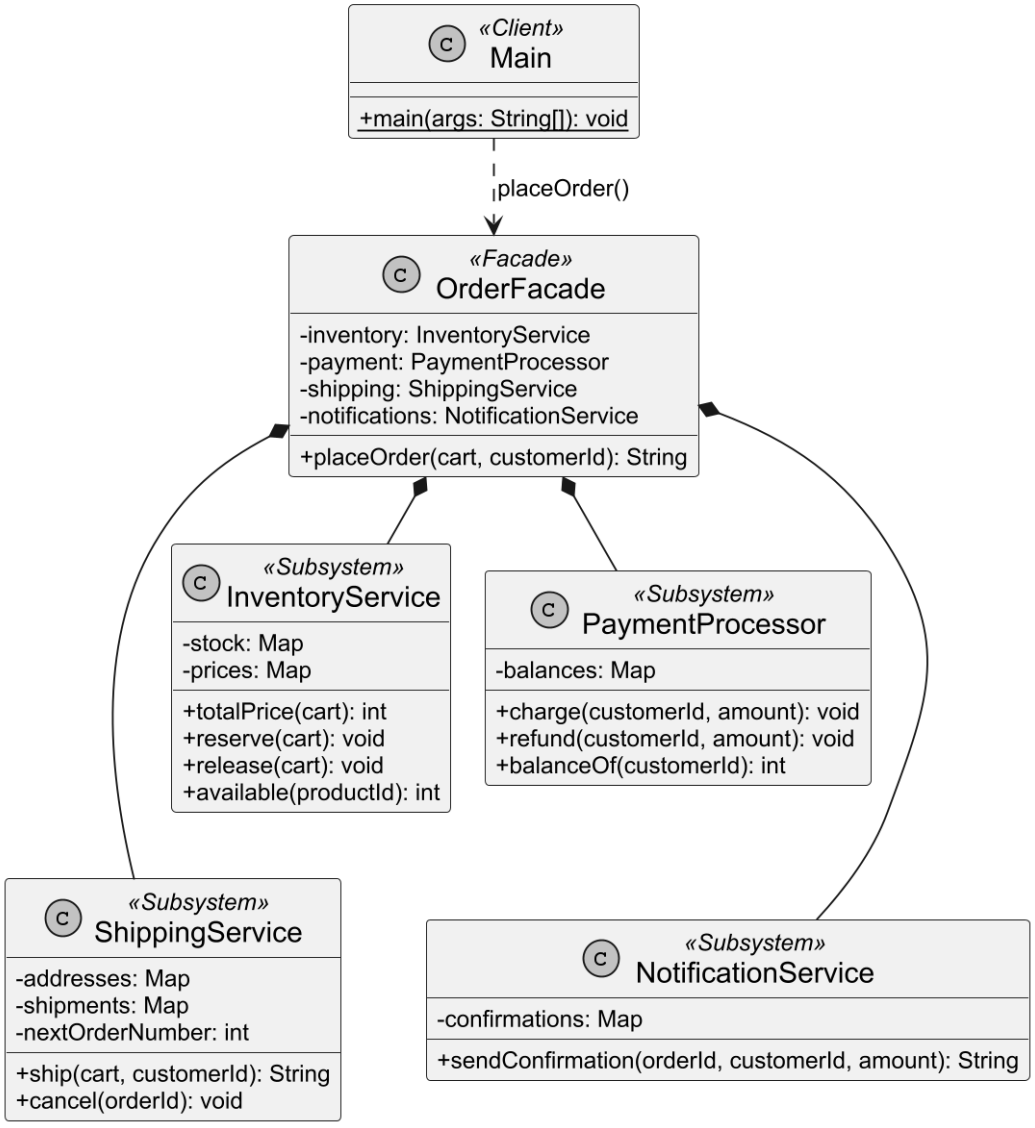


Figure 1. PlantUML diagram of the final code. Main depends only on OrderFacade. The facade owns its four services. Parameter types inside the subsystem operations are omitted for readability; names and return types match the source. No subsystem refers back to the facade [3].

3. Clean Code examples

3.1 Meaningful names

facade/InventoryService.java (lines 10-11)

```
public int totalPrice(Map<String, Integer> cart) {
    validateCart(cart);
```

totalPrice and validateCart state their purpose. Product quantities and customer IDs are named explicitly, which makes the order workflow easier to follow.

3.2 Single Responsibility

facade/OrderFacade.java (lines 19-22)

```
payment.charge(customerId, amount);
paid = true;
orderId = shipping.ship(items, customerId);
return notifications.sendConfirmation(orderId, customerId, amount);
```

The facade delegates payment, shipment creation, and confirmation. Each service owns its logic and state; the facade coordinates the sequence.

3.3 No leaking of subsystem details

facade/OrderFacade.java (lines 6-6)

```
private final InventoryService inventory = new InventoryService();
```

facade/OrderFacade.java (lines 11-11)

```
public String placeOrder(Map<String, Integer> cart, String customerId) {
```

The services are private implementation details. The public API accepts a cart and customer ID and returns a confirmation string. Main does not receive or construct subsystem objects.

3.4 Small focused methods

facade/PaymentProcessor.java (lines 10-16)

```
public void charge(String customerId, int amount) {
    validateAmount(amount);
    int balance = balanceOf(customerId);
    if (balance < amount) {
        throw new IllegalStateException("Insufficient balance for " + customerId + ".");
    }
    balances.put(customerId, balance - amount);
```

charge validates the amount, checks the balance, and performs one debit. Refunds are handled by a separate method. Private balances cannot be changed directly by the client.

3.5 Proper error handling

facade/OrderFacade.java (lines 23-31)

```
} catch (IllegalArgumentException | IllegalStateException exception) {
    if (orderId != null) {
        shipping.cancel(orderId);
    }
    if (paid) {
        payment.refund(customerId, amount);
    }
    inventory.release(items);
    throw exception;
```

Rollback reverses the completed steps: cancel an existing shipment, refund only a completed payment, and release reserved stock. The original exception reaches Main, which prints a clear failure message and continues the demonstration.

4. Demonstration and conclusion

Main uses the same facade and the same cart, containing the only available book, for three attempts. This demonstrates both the successful workflow and rollback without the client calling any service directly.

Customer	Result and state
LOW_BALANCE	Payment is declined. The reserved book is released; no shipment or confirmation is created.
NO_ADDRESS	Payment succeeds but shipping fails. The payment is refunded and the book is released.
CUSTOMER	The order succeeds. Stock decreases from 1 to 0, the balance decreases from 5000 to 4000 cents, and shipment ORD-1 and its confirmation are recorded.

Actual console output from the JDK 17 run:

```
Customer: LOW_BALANCE
Order failed: Insufficient balance for LOW_BALANCE.
Customer: NO_ADDRESS
Order failed: No delivery address for NO_ADDRESS.
Customer: CUSTOMER
Order ORD-1 confirmed for CUSTOMER, total: 10.00.
```

The facade hides service initialization, call ordering, and rollback coordination. The four subsystem classes remain public and unaware of the facade. Advanced clients can construct and call them directly, but then must manage their own workflow and recovery. The demonstration client deliberately uses only the facade.

The program compiled with JDK 17. Verification covered multi-item reservations, invalid carts, payment and shipping failures, and rollback after a confirmation failure. A trade-off is that OrderFacade depends on all four services; unrelated functionality should stay outside this API.

5. GitHub repository

https://github.com/Mikosh001/Assignment_4_Facade_Pattern

References

[1] Astana IT University. Assignment 4 - Facade Pattern instructions, 2026-2027.
[2] Refactoring.Guru. Facade.
[3] PlantUML. Class diagram.
[4] OpenAI Codex. Assistance with implementation, report drafting, and verification.